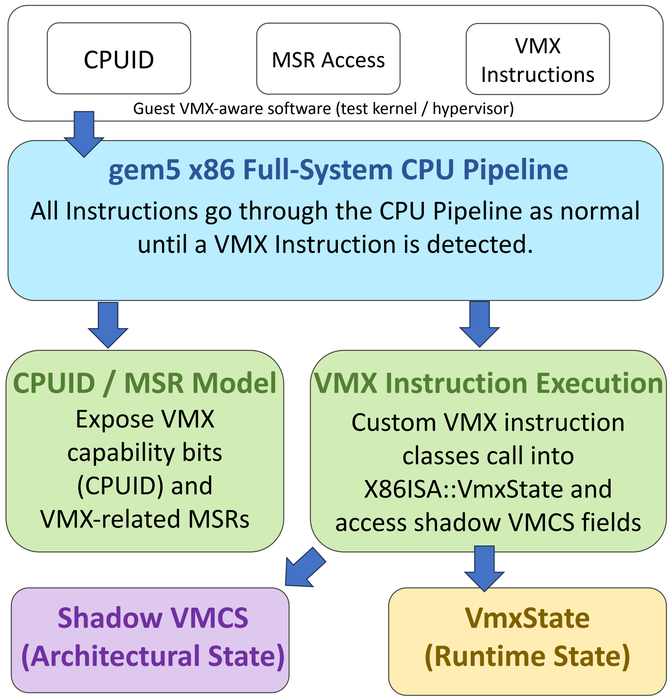
Intel VMX Support for gem5 Full-System Virtualization Research: A First Step

Seaver Olson Advisor: Dr Neil Kligensmith
Department of Computer Science, Loyola University Chicago

Problem Statement

- Modern CPUs rely on hardware virtualization software (i.e., VMX for Intel Processors)
- VMX reduces virtualization overhead by handling VMM-guest separation in hardware
- gem5 currently lacks VMX modeling
- This limits research on hypervisors / OS interactions

Architecture View



References

Intel Corporation. Intel® 64 and IA-32 Architectures Software Developer's Manual, Vols. 2C and 3C, VMX instruction and virtualization chapters, 2026.

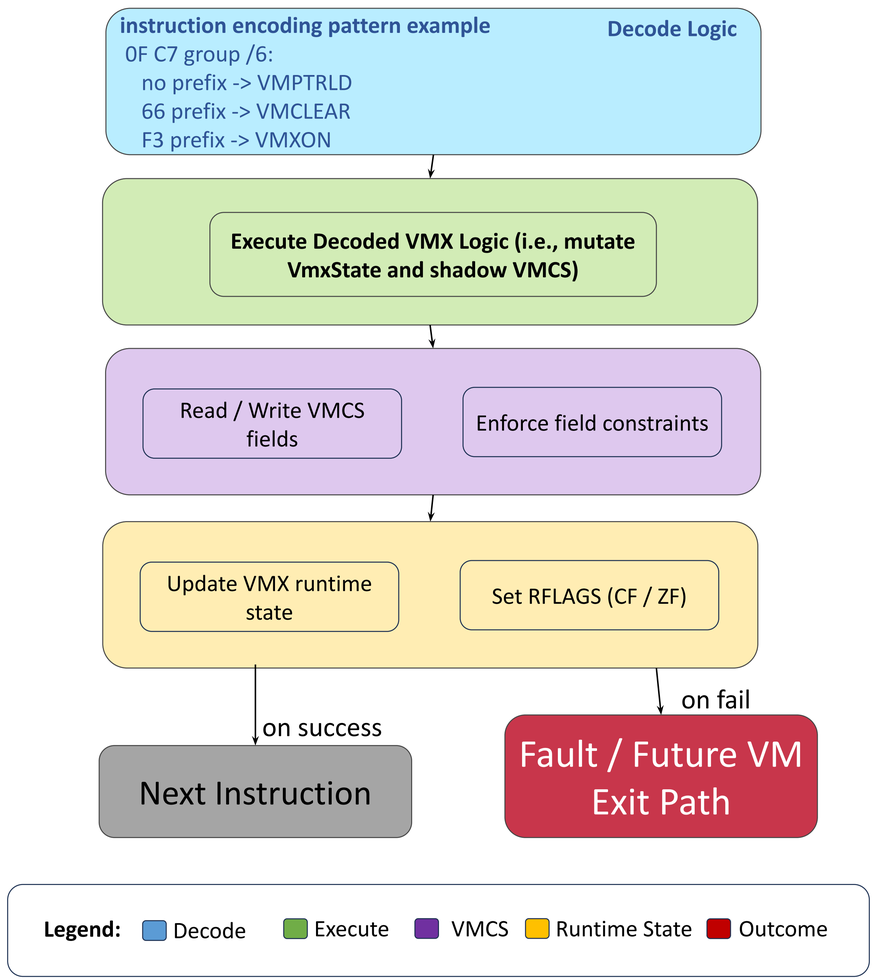
N. Binkert et al. "The gem5 Simulator." ACM SIGARCH Computer Architecture News, vol. 39, no. 2, pp. 1–7, 2011.

J. Lowe-Power et al. "The gem5 Simulator: Version 20.0+." arXiv:2007.03152, 2020.

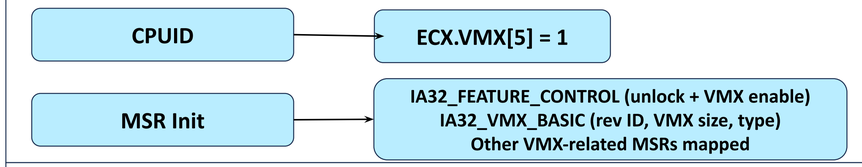
Key Idea / Contribution

I am extending gem5 with a functional model of Intel VMX for full-system virtualization :

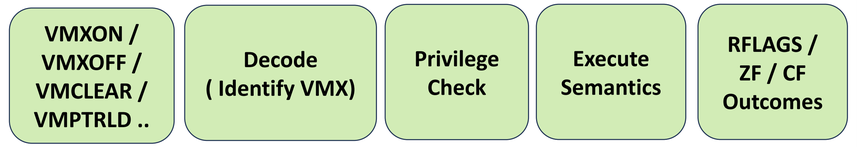
- Guest-visible VMX support via CPUID and MSR initialization
- Shadow VMCS model with typed fields and checkpointing
- ISA-level support for core VMX instructions (VMXON, VMWRITE, etc.)
- Correct privilege enforcement (CPL0) and RFLAGS-based outcomes
- Enables execution of VMX-aware software in gem5



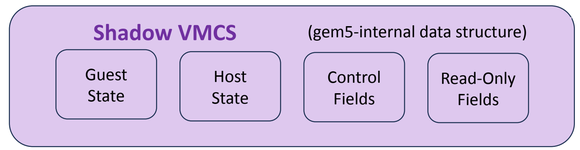
1. VMX Exposure (Architectural Discoverability)



2. VMX Instruction Support (Front-End + Outcomes)

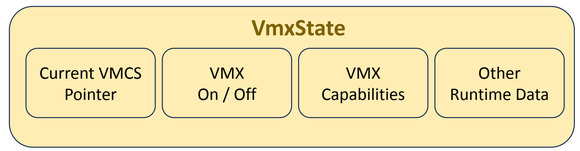


3. Shadow VMCS Model



- ✔ Typed fields (by width)
- ✔ Write permissions
- ✔ Value sanitization
- ✔ Serializable (checkpoint)

4. VMX Runtime State



- ✔ Managed by CPU
- ✔ Persists across checkpoints
- ✔ Integrated with gem5 CPU model

Current VMX Instructions

VMXON updates runtime VmxState: vmxActive, vmxonRegion, clears current VMCS pointer
VMCLEAR updates runtime current pointer if needed and clears/creates the shadow VMCS
VMPTRLD updates runtime currentVmcsPtr
VMPTRST writes the current VMCS pointer back to guest memory
VMWRITE writes a field into the Shadow VMCS
VMREAD reads from the shadow VMCS and writes the dest operand

Future Work

1. Complete control-flow instructions(i.e. VMLAUNCH, VMRESUME)
2. Implement full VM entry/exit state transitions
3. Populate VMCS exit-information fields from real execution events
4. Enforce VMX control constraints and extend MSR-based configs
5. Add memory virtualization support (EPT/VPID), integrate to MMU
6. Validate with guest workloads and a minimal VMX-enabled hypervisor